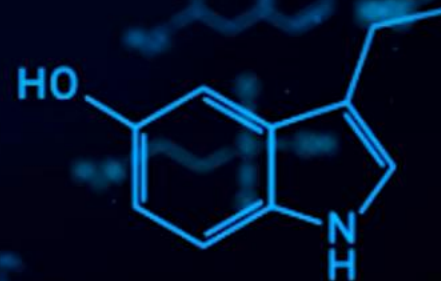


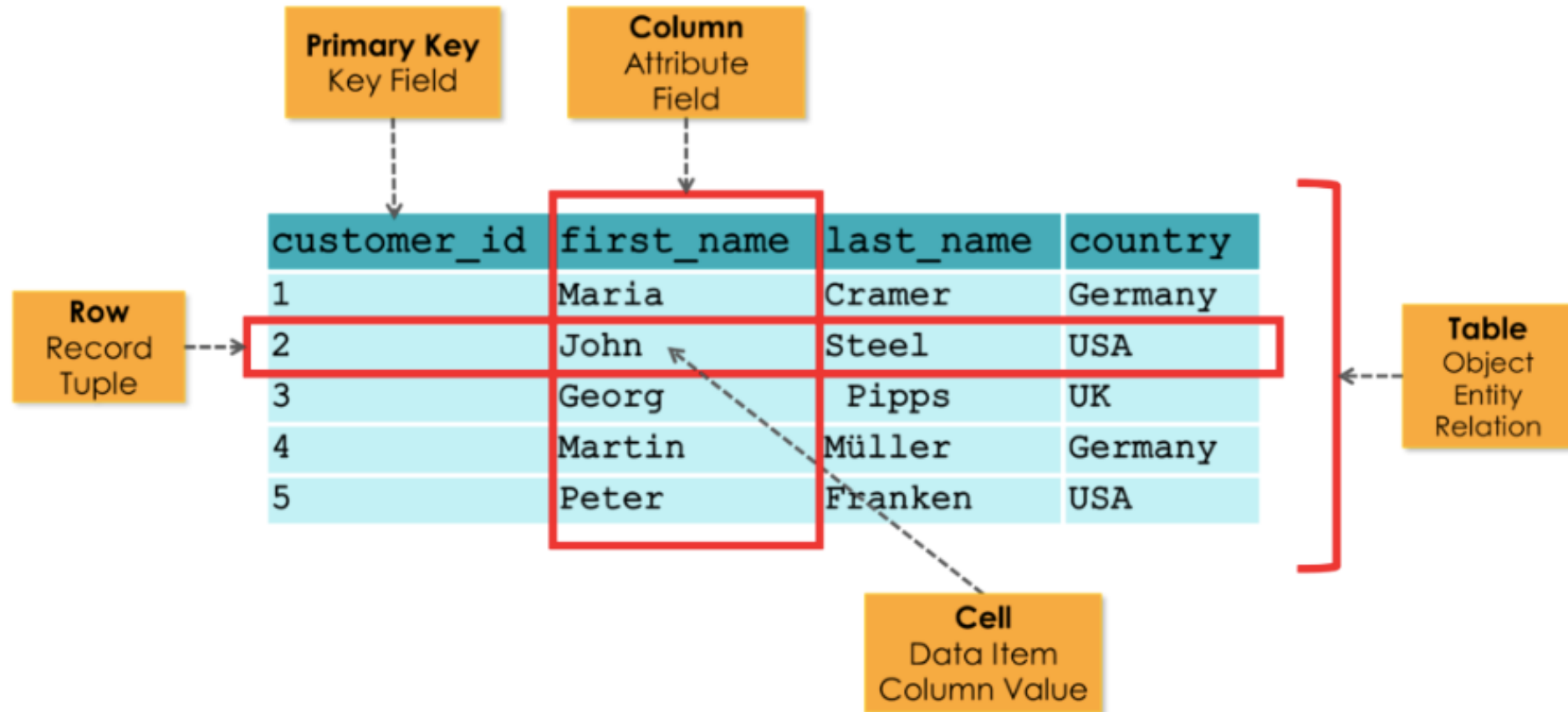
Basic SQL Commands: Using DQL and DML



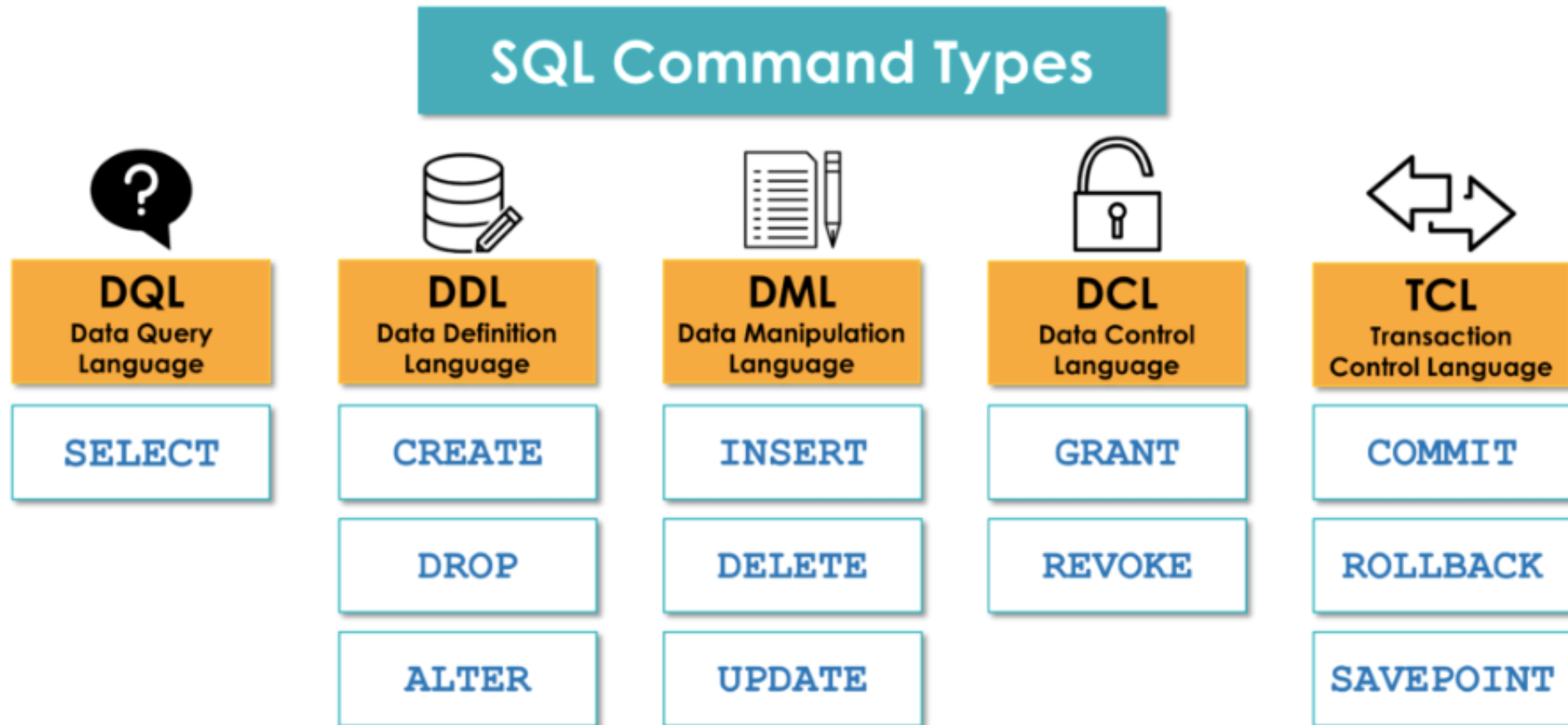
Objectives

- Review the fundamental concepts of SQL Commands
- Demonstrate how to use SQL aliases.
- Discuss how to use Data Query Language.
- Know the different SQL operators paired with the WHERE Clause.
- Discuss how to use Data Manipulation Language.

Review



SQL Commands



Data Query Language (DQL)

- **Data Query Language, (DQL)**, contains only one command called **SELECT**. It helps you to retrieve information from your database.
- **SELECT** is the most important command and concentrated focus of SQL. It contains many options and clauses which makes it very effective.

Data Manipulation Language (DML)

- **Data Manipulation Language, (DML)**, contains SQL commands used to manipulate your data inside your database.
- The **DML** command gives you tools to make changes to the records in a table, such as adding new records, deleting records, and updating records.
- The following commands considered to be the main DML commands:
 - **INSERT** — inserts data into a table.
 - **DELETE** — deletes data from a table.
 - **UPDATE** — updates existing data in a table.

ALIAS: Tables and Columns

Using the Schema name

```
105 • SELECT *
106 FROM DBInventory.Products;
107
```

100%	↺	15:103	
Result Grid   Filter Rows: Search			
	ProductID	Produ...	UnitPrice
▶	101	Laptop	500
▢	102	Mouse	20
	103	Keyb...	30
▢	104	Printer	100
	105	AVR	150
▢	105	AVR	150
	106	Mous...	5

What is an alias?

- SQL aliases are used to give a table, or a column in a table, a temporary name.
- Aliases are often used to make column names more readable.
- An alias only exists for the duration of that query.
- An alias is created with the AS keyword.

Note: In MySQL, you may or may not use “AS” when using aliases.

Example

```
SELECT ProductID as PI,  
       ProductName PN,  
       UnitPrice as UP  
from Products as P;
```

Result Grid   Filter Row				
	PI	PN	UP	
	101	Laptop	500	
▶	102	Mouse	20	
	103	Keyboard	30	
	104	Printer	100	
	105	AVR	150	
	105	AVR	150	
	106	Mousepad	5	

Use an ALIAS to:

1. Change a column or table name for improved readability within your code;
2. Change the output given to the recipients to address any formatting concerns.

USING SELECT STATEMENT

The MySQL SELECT Statement

- The SELECT statement is used to select data from a database.
- The data returned is stored in a result table, called the ***result-set***.

The MySQL SELECT Statement

Syntax:

```
SELECT column1, column2, ...  
FROM table_name;
```

Here, *column1, column2, ...* are the field names of the table you want to select data from. If you want to select all the fields available in the table, use the following syntax:

```
SELECT * FROM table_name;
```

The MySQL SELECT Statement

```
29 • SELECT * FROM tbl_studentInfo;
```

100% 8:25

Result Grid Filter Rows: Search Export:

	studentID	studentName	Gender	Course	yearLevel
▶	1001	Juan dela Cruz	Male	Course	1

tbl_studentInfo 1x Read Only

The MySQL SELECT Statement

```
46 • SELECT DepartmentID,  
47         DepartmentName,  
48         DepartmentLocation  
49 FROM DBHR.Department;
```

00%  23:41

Result Grid   Filter Rows:

	DepartmentID	DepartmentNa...	DepartmentLoca
▶	D01	HR	New York
	D02	ICT	San Francisco
	D03	Jane Marketing	Los Angeles
	NULL	NULL	NULL

Inserting Data

MySQL INSERT INTO Statement

- The INSERT INTO statement is used to insert new records in a table. It is possible to write the INSERT INTO statement in two ways:

1. Specify both the column names and the values to be inserted:

2. If you are adding values for all the columns of the table, you do not need to specify the column names in the SQL query. However, make sure the order of the values is in the same order as the columns in the table.

MySQL INSERT INTO Statement

Syntax:

- **INSERT INTO** *table_name*
VALUES (*value1*, *value2*, *value3*, ...);

MySQL INSERT INTO Statement

- The following SQL statement inserts a new record in the "tbl_studentInfo" table:

```
• INSERT INTO tbl_studentInfo (  
    studentID,  
    studentName,  
    Gender,  
    Course,  
    yearLevel)  
  
    VALUES (  
        '1001',  
        'Juan dela Cruz',  
        'Male',  
        'Course',  
        '1');
```

MySQL INSERT INTO Statement

- The following SQL statement inserts a new record in the "Department" table:

```
2 • INSERT INTO `DBHR`.`Department`  
3   ( `DepartmentID`,  
4     `DepartmentName`,  
5     `DepartmentLocation` )  
6   VALUES  
7   ( 'D01',  
8     'HR',  
9     'New York' );  
10  
11 • INSERT INTO `DBHR`.`Department`  
12   ( `DepartmentID`,  
13     `DepartmentName`,  
14     `DepartmentLocation` )  
15   VALUES  
16     ( 'D02', 'ICT', 'San Francisco' ),  
17     ( 'D03', 'Jane Marketing', 'Los Angeles' );  
18
```

MySQL INSERT INTO Statement

- The following SQL statement inserts a new record in the "Department" table:

```
INSERT INTO `DBHR`.`Employee`  
(`EmployeeID`,  
`EmployeeName`,  
`DepartmentID`)  
VALUES  
(101, 'Juan Cruz', 'D01'),  
(102, 'Ana Lopez', 'D01'),  
(103, 'Christine Medina', 'D02'),  
(104, 'Diane Perez', 'D02'),  
(105, 'Jessa Mendez', 'D03');
```

MySQL INSERT INTO Statement

- The following SQL statement inserts a new record in the "Employee" table:

```
INSERT INTO DBHR.Employee
(EmployeeID,
EmployeeName,
DepartmentID)
VALUES
(106, 'Harry Potter', 'D01'),
(107, 'Hermione Granger', 'D02'),
(108, 'Ron Weasley', 'D02');
```

How would you know if record
was successfully inserted?

By using the SELECT STATEMENT

Example:

```
SELECT * FROM DBHR.Employee;
```

```
34 • INSERT INTO DBHR.Employee
35   (EmployeeID,
36   EmployeeName,
37   DepartmentID)
38   VALUES
39   (106, 'Harry Potter', 'D01'),
40   (107, 'Hermione Granger', 'D02'),
41   (108, 'Ron Weasley', 'D02');
42
43
44 • Select * FROM DBHR.Employee;
45
```

100%  27:31

Result Grid   Filter Rows:

	EmployeeID	EmployeeName	DepartmentID
	101	Juan Cruz	D01
	102	Ana Lopez	D01
	103	Christine Medina	D02
▶	104	Diane Perez	D02
	105	Jessa Mendez	D03
	106	Harry Potter	D01
	107	Hermione Granger	D02
	108	Ron Weasley	D02

Operators in The WHERE Clause

- You can use operators other than the = operator to filter the search.
- Example
- Select all students with a StudentID greater than 4001:
- `SELECT * FROM Students
WHERE StudentID > 4001;`

Operators in The WHERE Clause

The following operators can be used in the WHERE clause:

Operator	Description
=	Equal
>	Greater than
<	Less than
>=	Greater than or equal
<=	Less than or equal
<>	Not equal. Note: In some versions of SQL this operator may be written as !=
BETWEEN	Between a certain range
LIKE	Search for a pattern
IN	To specify multiple possible values for a column

Updating Records

The SQL UPDATE STATEMENT

The UPDATE statement is used to modify the existing records in a table.

- UPDATE Syntax

```
UPDATE table_name  
SET column1 = value1, column2 = value2, ...  
WHERE condition;
```

The SQL UPDATE STATEMENT

```
34 • INSERT INTO DBHR.Employee
35   (EmployeeID,
36   EmployeeName,
37   DepartmentID)
38   VALUES
39   (106, 'Harry Potter', 'D01'),
40   (107, 'Hermione Granger', 'D02'),
41   (108, 'Ron Weasley', 'D02');
42
43 • SELECT * FROM DBHR.Employee;
44
```

00% 1:33 1 error found

Result Grid Filter Rows: Search

EmployeeID	EmployeeName	DepartmentID
101	Juan Cruz	D01
102	Ana Lopez	D01
103	Christine Medina	D02
104	Diane Perez	D02
105	Jessa Mendez	D03
106	Harry Potter	D01
107	Hermione Granger	D02
108	Ron Weasley	D02
NULL	NULL	NULL



```
50  -- -----USING UPDATE STATEMENT -----
51 • UPDATE DBHR.Employee
52   SET EmployeeName = 'Hermione Weasley'
53   WHERE EmployeeID=107;
54
55 • SELECT * FROM DBHR.Employee;
56
```

00% 29:57

Result Grid Filter Rows: Search Edit: E

EmployeeID	EmployeeName	DepartmentID
101	Juan Cruz	D01
102	Ana Lopez	D01
103	Christine Medina	D02
104	Diane Perez	D02
105	Jessa Mendez	D03
106	Harry Potter	D01
107	Hermione Weasley	D02
108	Ron Weasley	D02
NULL	NULL	NULL

UPDATE Multiple Records

- It is the **WHERE** clause that determines how many records will be updated.
- The following SQL statement will update the OvertimeRate to 600 for all records where OvertimeHours is greater than or equal to 10.

57 • `SELECT * FROM DBHR.Payroll;`

58

100% 21:51

Result Grid Filter Rows: Search Edit:

	Payroll_ID	BasicSalary	OvertimeHou...	OvertimeRate	Deduction	Employ
▶	1	20000	10	450	150	101
	2	28000	5	525	200	102
	3	45000	8	525	250	103
	4	28000	12	300	150	104
	5	40000	15	250	180	105
	NULL	NULL	NULL	NULL	NULL	NULL

UPDATE Multiple Records

57 • **SELECT** * **FROM** DBHR.Payroll;

58

100% 21:51

Result Grid Filter Rows: Search Edit:

	Payroll_ID	BasicSalary	OvertimeHou...	OvertimeRate	Deduction	Employ
▶	1	20000	10	450	150	101
	2	28000	5	525	200	102
	3	45000	8	525	250	103
	4	28000	12	300	150	104
	5	40000	15	250	180	105
	NULL	NULL	NULL	NULL	NULL	NULL



60 -- -----USING UPDATE STATEMENT MULTIPLE RECORDS---

61 • **UPDATE** DBHR.Payroll

62 **SET** OvertimeRate = 600

63 **WHERE** OvertimeHours>=10;

64

65 • **SELECT** * **FROM** DBHR.Payroll;

66

100% 28:57

Result Grid Filter Rows: Search Edit: Export/Import

	Payroll_ID	BasicSalary	OvertimeHou...	OvertimeRate	Deduction	EmployeeID
▶	1	20000	10	600	150	101
	2	28000	5	525	200	102
	3	45000	8	525	250	103
	4	28000	12	600	150	104
	5	40000	15	600	180	105
	NULL	NULL	NULL	NULL	NULL	NULL

UPDATE Warning

Be careful when updating records.

If you omit the WHERE clause, ALL records will be updated

UPDATE WARNING: Multiple Records

```
60  -- -----USING UPDATE STATEMENT MULTIPLE RECORDS-----
61  •  UPDATE DBHR.Payroll
62      SET OvertimeRate = 600
63      WHERE OvertimeHours>=10;
64
65  •  SELECT * FROM DBHR.Payroll;
```

66 00% 28:57

Result Grid Filter Rows: Search Edit: Export/Import

Payroll_ID	BasicSalary	OvertimeHou...	OvertimeRate	Deduction	EmployeeID
1	20000	10	600	150	101
2	28000	5	525	200	102
3	45000	8	525	250	103
4	28000	12	600	150	104
5	40000	15	600	180	105
NULL	NULL	NULL	NULL	NULL	NULL



```
60  -- -----USING UPDATE STATEMENT MULTIPLE RECORDS-----
61  •  UPDATE DBHR.Payroll
62      SET OvertimeRate = 600;
63
64  •  SELECT * FROM DBHR.Payroll;
```

65 00% 1:56

Result Grid Filter Rows: Search Edit: Export/Import

Payroll_ID	BasicSalary	OvertimeHou...	OvertimeRate	Deduction	EmployeeID
1	20000	10	600	150	101
2	28000	5	600	200	102
3	45000	8	600	250	103
4	28000	12	600	150	104
5	40000	15	600	180	105
NULL	NULL	NULL	NULL	NULL	NULL

DELETING RECORDS

SQL DELETE Statement

- The DELETE statement is used to delete existing records in a table.

DELETE Syntax:

```
DELETE FROM table_name WHERE condition;
```

SQL DELETE Statement

```
64 • SELECT * FROM DBHR.Payroll;
```

```
65
```

100% 8:67

Result Grid

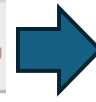


Filter Rows:



Search

Edit:



	Payroll_ID	BasicSalary	OvertimeHou...	OvertimeRate	Deduction	EmployeeID
▶	1	20000	10	600	150	101
▢	2	28000	5	600	200	102
▢	3	45000	8	600	250	103
▢	4	28000	12	600	150	104
▢	5	40000	15	600	180	105
▢	NULL	NULL	NULL	NULL	NULL	NULL

```
66  -- -----USING DELETE STATEMENT -----
```

```
67 • DELETE FROM DBHR.Payroll
```

```
68 WHERE Deduction=180;
```

```
69
```

```
70 • SELECT * FROM DBHR.Payroll;
```

```
71
```

100% 25:64

Result Grid



Filter Rows:



Search

Edit:



	Payroll_ID	BasicSalary	OvertimeHou...	OvertimeRate	Deduction	Employee
▶	1	20000	10	600	150	101
▢	2	28000	5	600	200	102
▢	3	45000	8	600	250	103
▢	4	28000	12	600	150	104
▢	NULL	NULL	NULL	NULL	NULL	NULL

Delete All Records

- It is possible to delete all rows in a table without deleting the table.
- This means that the table structure, attributes, and indexes will be intact:

```
DELETE FROM table_name;
```

Delete All Records

```
66  -- -----USING DELETE STATEMENT -----
67 •  DELETE FROM DBHR.Payroll
68  WHERE Deduction=180;
69
70 •  SELECT * FROM DBHR.Payroll;
71
```

100% 25:64

Result Grid Filter Rows: Search Edit:

	Payroll_ID	BasicSalary	OvertimeHou...	OvertimeRate	Deduction	Employee
▶	1	20000	10	600	150	101
	2	28000	5	600	200	102
	3	45000	8	600	250	103
	4	28000	12	600	150	104
	NULL	NULL	NULL	NULL	NULL	NULL



```
70 •  SELECT * FROM DBHR.Payroll;
71
72 •  DELETE FROM DBHR.Payroll;
73
74 •  SELECT * FROM DBHR.Payroll;
75
```

100% 28:64

Result Grid Filter Rows: Search Edit:

	Payroll_ID	BasicSalary	OvertimeHou...	OvertimeRate	Deduction	EmployeeID
▶	NULL	NULL	NULL	NULL	NULL	NULL

Delete a Table

- To delete the table completely, use the DROP TABLE statement:
- `DROP TABLE DBHR.Payroll;`